AI Pathfinding Visualizer

Overview :

This project is an interactive **Pathfinding Visualizer** that demonstrates how fundamental Artificial Intelligence algorithms compute optimal paths in a grid-based environment.

It allows users to simulate real-world navigation scenarios and observe how different algorithms explore and find paths between two points.

The project is built as part of a **Bring Your Own Project (BYOP)** to apply AI/ML fundamentals in a practical, visual, and intuitive way.

PROBLEM STATEMENT :

In real-world environments such as urban navigation (e.g., road networks in cities), finding the shortest or most efficient route is a critical challenge.

Most users rely on applications like GPS systems without understanding how these decisions are made.

This project addresses the problem:

How can we simulate and visualize intelligent pathfinding to better understand decision-making in AI systems?

FEATURES AND FUNCTIONAL REQUIREMENTS :

Functional Features:

- Interactive grid-based environment
- User-defined:
  - Start node
  - End node
  - Obstacles (walls)
- Visualization of algorithms in real time
- Multiple algorithm support:
  - Breadth-First Search (BFS)
  - Depth-First Search (DFS)
  - A* (A-Star) Algorithm
- Path reconstruction after goal is reached

Controls :

- Mouse Click → Place start, end, and barriers
- `1` → BFS
- `2` → DFS
- `3` → A*
- `SPACE` → Run algorithm
- `C` → Clear grid

TECHNOLOGY STACK AND ARCHITECTURE :

- **Language:** Python
- **Library:** Pygame
- **Concepts:** AI Search Algorithms, Heuristics

Architecture :

- Grid-based system (2D matrix)
- Each cell represented as a node (Spot class)
- Algorithms operate on graph abstraction:

- Nodes → grid cells
- Edges → adjacent cells

Flow :

1. User creates environment
2. Grid is converted into graph
3. Selected algorithm runs
4. Visualization updates in real time

STEPS TO INSTALL :

Step 1 : Clone Repository

```
git clone https://github.com/Harsh-872/AI-Pathfinder-Visualizer.git
cd pathfinding-visualizer
```

Step 2 : Install Dependencies

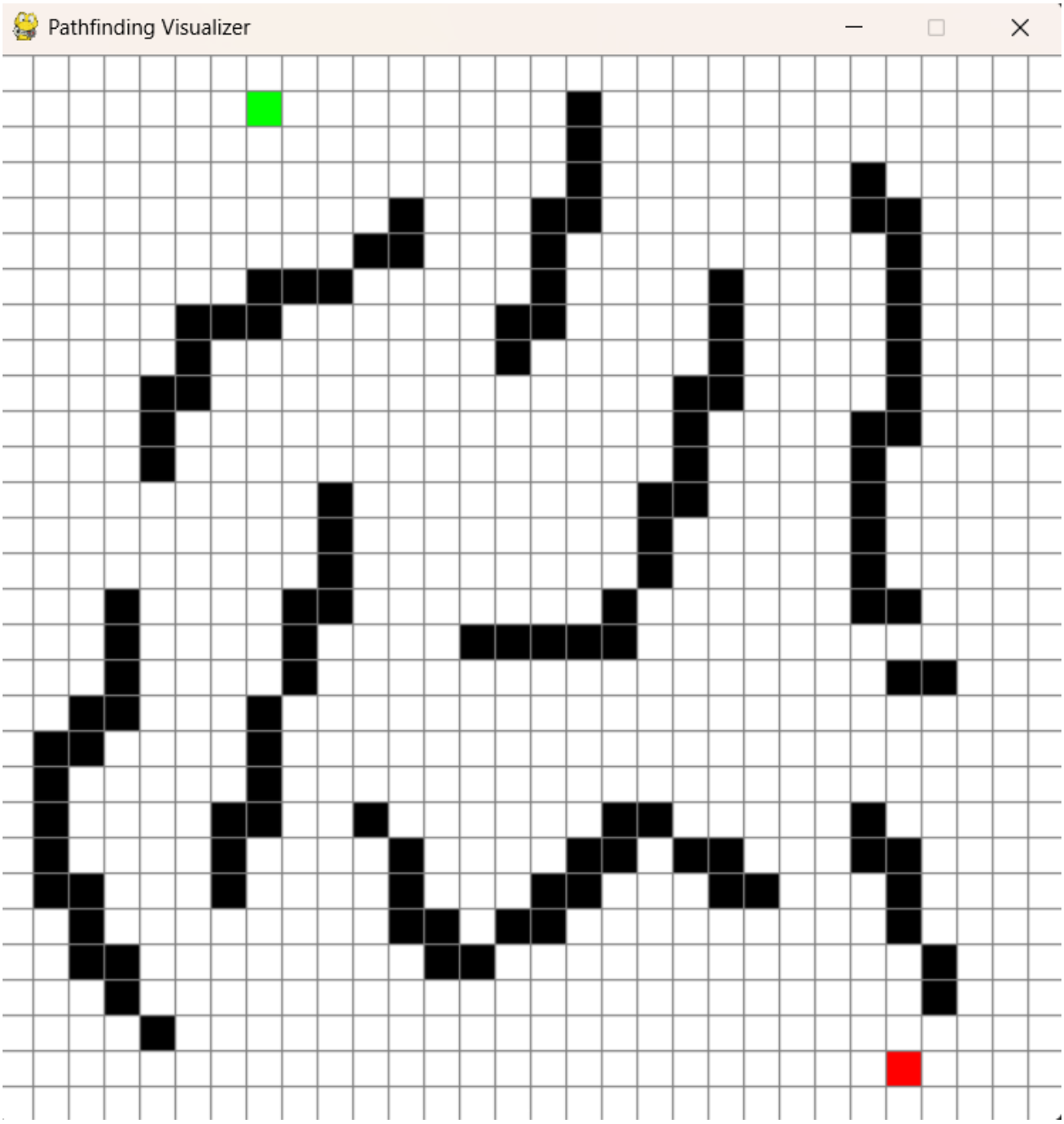
```
pip install pygame
```

Step 3 : Run Project

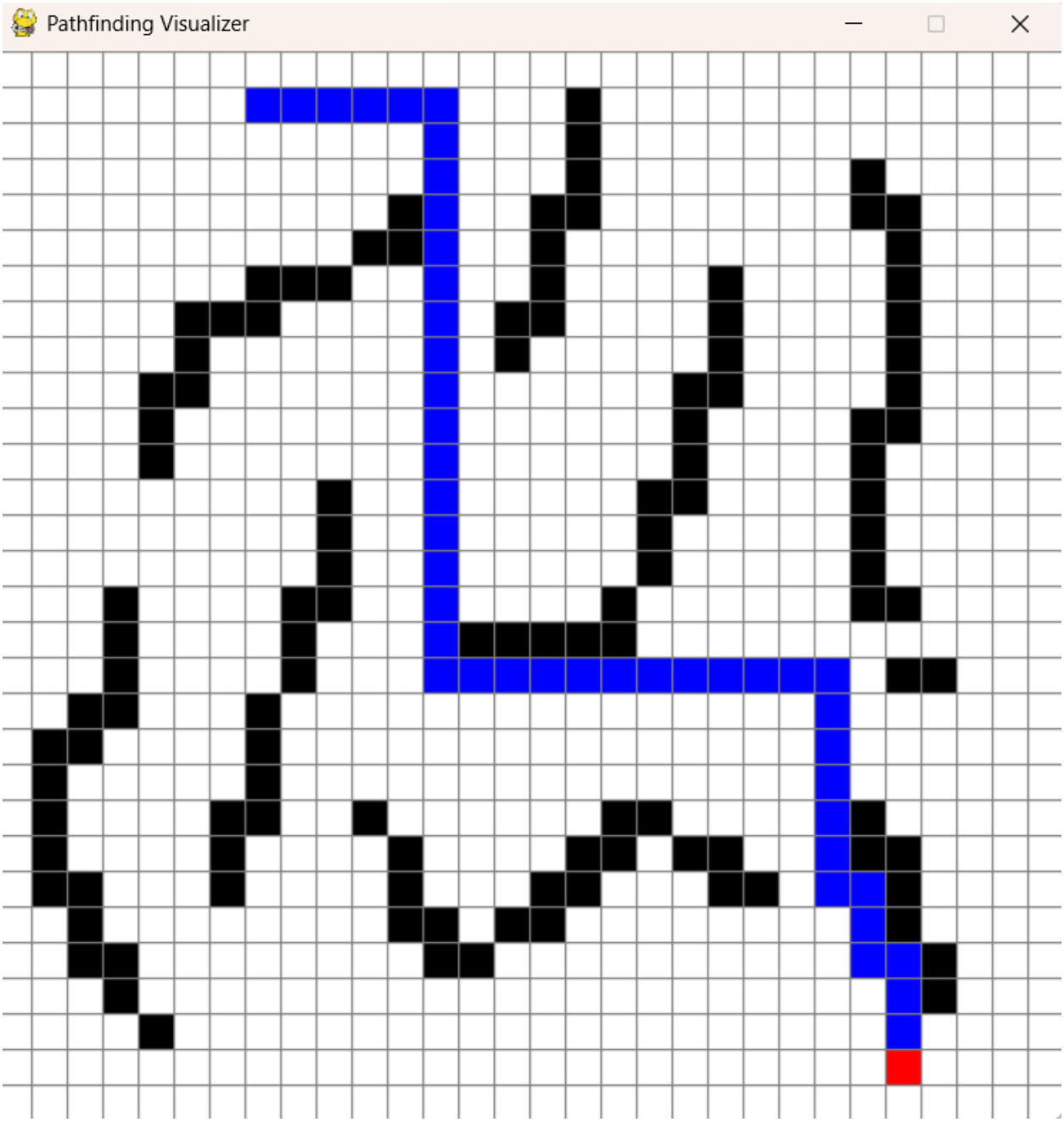
```
python main.py
```

SCREENSHOTS :

1. Grid Setup :



2. Final Path Output :



TESTING APPROACH :

Functional Testing:

- Verified pathfinding for all algorithms
- Checked correct handling of:
  - Obstacles
  - Start/End placement
- Tested edge cases:
  - No possible path
  - Start = End

Algorithm Validation:

- BFS → Always finds shortest path
- DFS → May not find optimal path
- A* → Efficient and optimal

User Interaction Testing:

- Mouse clicks
- Keyboard inputs
- Grid reset functionality

NON-FUNCTIONAL REQUIREMENTS :

- **Performance:** Real-time visualization with smooth updates
- **Usability:** Simple controls and intuitive interface
- **Scalability:** Grid size can be modified
- **Maintainability:** Modular and readable code structure
- **Reliability:** Handles invalid inputs gracefully

FUTURE ENHANCEMENTS :

- Add **Dijkstra's Algorithm**
- Introduce **weighted nodes (traffic simulation)**
- GUI buttons instead of keyboard controls
- Real-world map integration
- Performance metrics (time taken, nodes explored)
- 3D visualization

LEARNING OUTCOMES :

- Understanding of search algorithms
- Importance of heuristics in AI
- Differences between informed and uninformed search
- Practical implementation of theoretical concepts

AUTHOR :

- **Name :** Tanish Paharia
- **Department :** BTech CSE Core
- **Course :** Fundamentals in AI & ML
- **Registration No. :** 25BCE10219
- **Institute :** VIT Bhopal

FINAL NOTE :

This project emphasizes **clarity of concepts over complexity**, demonstrating how AI techniques can solve real-world problems effectively.